S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 11
Postlab (OPEN ENDED PRACTICAL)

Aim: To write and execute a C program to construct and display a **Resource Allocation Graph (RAG)** by taking the allocation and request matrices as input, and to identify the allocation edges and request edges

Name: Siddhanth Navin Sukhdeve

USN: CM24048

Semester / Year: IV Semester / II Year

Academic Session: 2025 – 2026

Date of Performance:

Date of Submission:

Aim: To write and execute a C program to construct and display a **Resource Allocation Graph (RAG)** by taking the allocation and request matrices as input, and to identify the allocation edges and request edges

❖ **Objectives:**

- To understand the concept of a **Resource Allocation Graph (RAG)** in operating systems.
- To identify and display allocation edges (Resource → Process) and request edges.
- To analyze how resource allocation and requests can be used to study system.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz
- RAM: 512 MB or higher
- Storage: 100 MB free space

✓ **Software Requirements:**

- Operating System: Linux/Unix-based
- Shell: Bash 4.0 or higher
- Text Editor: Nano, Vim, or any preferred editor

❖ **Theory:**

What is Resource Allocation Graph (RAG)?

A Resource Allocation Graph (RAG) is a graphical representation used in operating systems to depict the allocation of resources to processes and the requests made by processes for resources. It helps in understanding the state of resource distribution in a system and is mainly used for **deadlock detection and analysis**.

In a Resource Allocation Graph, there are two types of nodes:

- **Process nodes (P1, P2, …)**
- **Resource nodes (R1, R2, …)**

The graph uses directed edges to represent relationships between processes and resources.

Types of Edges in Resource Allocation Graph:

- **Allocation Edge (Resource → Process):**
  This edge shows that a resource has been allocated to a process. Example: R1 → P1 means resource R1 is allocated to process P1.
- **Request Edge (Process → Resource):**
  This edge shows that a process is requesting a resource. Example: P2 → R3 means process P2 is requesting resource R3.

Working of Resource Allocation Graph

The Resource Allocation Graph works based on the current allocation and request state of processes and resources. The following steps explain its working:

- **Input Allocation Matrix:**
  The allocation matrix represents which resources are currently assigned to which processes.
- **Input Request Matrix:**
  The request matrix represents which resources are being requested by processes.
- **Graph Construction:**
  - For every entry allocation[i][j] = 1, an edge is drawn from resource Rj to process Pi.
  - For every entry request[i][j] = 1, an edge is drawn from process Pi to resource Rj.
- **Edge Representation:**
  The program displays all allocation and request edges, forming a complete Resource Allocation Graph.
- **Deadlock Analysis (Conceptual):**
  If the graph contains a **cycle**, there is a possibility of deadlock (especially when each resource has a single instance).

Example of Resource Allocation Graph

Suppose there are:

- 2 Processes: P0, P1
- 2 Resources: R0, R1

If:

- P0 holds R0 → Edge: R0 → P0
- P1 requests R0 → Edge: P1 → R0

This forms a dependency that can be visualized using the graph.

Advantages of Resource Allocation Graph

- Helps in visualizing resource usage in a system.
- Useful for detecting deadlocks in simple cases.
- Provides clear representation of process-resource relationships.
- Easy to implement using matrices in programming.

Disadvantages of Resource Allocation Graph

- Becomes complex when the number of processes and resources increases.
- Not efficient for systems with multiple instances of resources.
- Requires additional algorithms for complete deadlock detection.
- Graph analysis can be difficult for large-scale systems.

Applications of Resource Allocation Graph

- Deadlock detection in operating systems.
- Resource management analysis.
- Understanding process synchronization.
- Teaching and simulation of OS concepts.

❖ **CODE:**

```
  #include <stdio.h>
#define MAX 10

int main() {
    int p, r;

    printf("Enter number of processes: ");
    scanf("%d", &p);

    printf("Enter number of resources: ");
    scanf("%d", &r);

    int allocation[MAX][MAX], request[MAX][MAX];
    printf("\nEnter Allocation Matrix:\n");
    for (int i = 0; i < p; i++) {
        for (int j = 0; j < r; j++) {
```

```
            scanf("%d", &allocation[i][j]);
        }
    }
    // Step 2: Input Request Matrix
    printf("\nEnter Request Matrix:\n");
    for (int i = 0; i < p; i++) {
        for (int j = 0; j < r; j++) {
            scanf("%d", &request[i][j]);
        }
    }
    printf("\n============================\n");
    printf("RESOURCE ALLOCATION GRAPH\n");
    printf("============================\n");
    // Step 3: Show Allocation Edges
    printf("\n Checking Allocation Edges (Resource -> Process)\n");
    for (int i = 0; i < p; i++) {
        for (int j = 0; j < r; j++) {
            printf("Checking allocation[%d][%d] = %d\n", i, j, allocation[i][j]);
            if (allocation[i][j] == 1) {
                printf("=> Resource R%d is allocated to Process P%d\n", j, i);
                printf("   Edge: R%d --> P%d\n", j, i);
            }
        }
    }
    // Step 4: Show Request Edges
    printf("\nChecking Request Edges (Process -> Resource)\n");
    for (int i = 0; i < p; i++) {
        for (int j = 0; j < r; j++) {
            printf("Checking request[%d][%d] = %d\n", i, j, request[i][j]);
            if (request[i][j] == 1) {
                printf("=> Process P%d is requesting Resource R%d\n", i, j);
                printf("   Edge: P%d --> R%d\n", i, j);
            }
        }
    }

    // Step 5: Summary
    printf("\nGraph Construction Completed!\n");
    printf("All allocation and request edges are shown above.\n");
    return 0;

}
```

❖ Output:

```
C:\Users\Siddhanth\OneDrive

Enter number of processes: 4
Enter number of resources: 3
Enter Allocation Matrix:
1 0 1
0 1 0
1 0 0
0 0 1

Enter Request Matrix:
0 1 0
1 0 1
0 1 1
1 0 0
```

```
============================
RESOURCE ALLOCATION GRAPH
============================

Checking Allocation Edges (Resource -> Process)
Checking allocation[0][0] = 1
=> Resource R0 is allocated to Process P0
   Edge: R0 --> P0
Checking allocation[0][1] = 0
Checking allocation[0][2] = 1
=> Resource R2 is allocated to Process P0
   Edge: R2 --> P0
Checking allocation[1][0] = 0
Checking allocation[1][1] = 1
=> Resource R1 is allocated to Process P1
   Edge: R1 --> P1
Checking allocation[1][2] = 0
Checking allocation[2][0] = 1
=> Resource R0 is allocated to Process P2
   Edge: R0 --> P2
Checking allocation[2][1] = 0
Checking allocation[2][2] = 0
Checking allocation[3][0] = 0
Checking allocation[3][1] = 0
Checking allocation[3][2] = 1
=> Resource R2 is allocated to Process P3
   Edge: R2 --> P3
```

```
Checking Request Edges (Process -> Resource)
Checking request[0][0] = 0
Checking request[0][1] = 1
=> Process P0 is requesting Resource R1
   Edge: P0 --> R1
Checking request[0][2] = 0
Checking request[1][0] = 1
=> Process P1 is requesting Resource R0
   Edge: P1 --> R0
Checking request[1][1] = 0
Checking request[1][2] = 1
=> Process P1 is requesting Resource R2
   Edge: P1 --> R2
Checking request[2][0] = 0
Checking request[2][1] = 1
=> Process P2 is requesting Resource R1
   Edge: P2 --> R1
Checking request[2][2] = 1
=> Process P2 is requesting Resource R2
   Edge: P2 --> R2
Checking request[3][0] = 1
=> Process P3 is requesting Resource R0
   Edge: P3 --> R0
Checking request[3][1] = 0
Checking request[3][2] = 0

Graph Construction Completed!
All allocation and request edges are shown above.
```

- **Conclusion**: The Resource Allocation Graph is an important concept in operating systems that helps in understanding how processes interact with resources. By analyzing allocation and request edges, it becomes easier to detect potential deadlocks and manage system resources efficiently.

- **Discussion Questions:**

  1. **What is a Resource Allocation Graph (RAG)?**
  2. **What do allocation edges represent in a RAG?**
  3. **What do request edges represent in a RAG?**
  4. **How are allocation and request matrices used in the program?**
  5. **How can a cycle in a RAG indicate a potential deadlock?**

- **References:**

https://www.geeksforgeeks.org/disk-scheduling-algorithms/
https://chatgpt.com/

Date:____/____/2026

Signature
Course Coordinator
B.Tech CSE(AIML)
Sem: 4 / 2025-26